

Credit Card Fraud Detection Using Machine Learning

Prepared by:

Amisha Chaudhari

Project Type: Data Science Portfolio Project

Tools and Technologies:

Python, pandas, NumPy, scikit-learn, Streamlit, Jupyter Notebook, GitHub

Project Focus:

Fraud Detection, Classification, Machine Learning, Financial Risk Analytics

Table of Contents

1. Executive Summary.....	(1)
2. Introduction.....	(2)
3. Business Problem.....	(3)
4. Dataset Overview.....	(4)
5. Exploratory Data Analysis.....	(5)
5.1 Missing Values.....	(5)
5.2 Duplicate Values.....	(5)
5.3 Class Distribution.....	(5)
5.4 Transaction Amount Analysis.....	(5)
5.5 Time-Based Analysis.....	(5)
6. Data Preprocessing.....	(6)
7. Model Development.....	(7)
8. Model Evaluation.....	(8)
8.1 Accuracy.....	(8)
8.2 Precision.....	(8)
8.3 Recall.....	(8)
8.4 F1-Score.....	(8)
8.5 Confusion Matrix.....	(8)
9. Streamlit Application.....	(9)
10. Business Impact.....	(10)
11. Limitations.....	(11)
12. Future Work.....	(12)
13. Conclusion.....	(13)
14. References.....	(14)

1.Executive Summary

This project focuses on detecting fraudulent credit card transactions using machine learning. Credit card fraud is a major challenge for banks, fintech companies, payment processors, and customers because fraudulent activity is rare, difficult to identify, and financially harmful.

The goal of this project is to build an end-to-end fraud detection system that can analyze transaction data and classify transactions as legitimate or fraudulent. The project includes exploratory data analysis, data preprocessing, model development, model evaluation, and a Streamlit-based prediction application.

The dataset contains 284,807 credit card transactions with anonymized numerical features, transaction time, transaction amount, and a target variable indicating whether each transaction is legitimate or fraudulent. One of the main challenges in this project is the strong class imbalance, where fraudulent transactions represent only a very small percentage of the total data.

The final solution demonstrates how machine learning can support fraud monitoring by identifying suspicious transactions and helping financial institutions reduce potential losses.

2.Introduction

Credit card fraud detection is an important application of machine learning in the financial industry. As digital payments continue to increase, financial institutions must identify suspicious transactions quickly and accurately. Fraudulent transactions can lead to financial losses, customer dissatisfaction, and security concerns.

Traditional rule-based fraud detection systems may not be flexible enough to detect new fraud patterns. Machine learning provides a more data-driven approach by learning patterns from historical transaction data and using those patterns to classify future transactions.

This project applies machine learning techniques to a credit card transaction dataset to detect fraudulent transactions. The project follows a practical data science workflow, beginning with data exploration and ending with a deployable Streamlit application for fraud prediction.

3.Business Problem

The business problem is to identify fraudulent credit card transactions from a large volume of legitimate transactions. Fraudulent transactions are rare, which makes this a difficult classification problem. A model that predicts every transaction as legitimate may still achieve high accuracy, but it would fail to detect fraud.

For financial institutions, the cost of missing a fraudulent transaction can be high. At the same time, incorrectly flagging legitimate transactions as fraud can create poor customer experience. Therefore, the goal is not only to build an accurate model, but also to evaluate the model using metrics that are meaningful for fraud detection.

The main business objective is to create a system that can help detect suspicious transactions, reduce financial risk, and support faster fraud investigations.

4.Dataset Overview

The dataset used in this project contains credit card transactions made by cardholders. The data includes anonymized numerical features, transaction time, transaction amount, and a target variable.

Dataset details:

- Total rows: 284,807
- Total columns: 31

Features:

- - Time: Time elapsed between transactions
- - V1 to V28: Anonymized numerical features created using PCA transformation
- - Amount: Transaction amount
- - Class: Target variable

Target variable:

- - Class 0: Legitimate transaction
- - Class 1: Fraudulent transaction

The dataset is highly imbalanced. Most transactions are legitimate, while only a small number are fraudulent. This imbalance is a key challenge in fraud detection because the model must learn to identify rare fraud cases.

Class	Meaning	Count
0	Legitimate Transaction	284,315
1	Fraudulent Transaction	492

Fraudulent transactions represent approximately 0.173% of the dataset. This shows that fraud detection is a rare-event classification problem.

5.Exploratory Data Analysis

Exploratory Data Analysis was performed to understand the structure, quality, and patterns in the transaction data. The analysis focused on class distribution, transaction amount behavior, missing values, duplicate records, and time-based transaction patterns.

5.1 Missing Values

The dataset was checked for missing values before model development. Missing value analysis is important because missing data can reduce model performance and create biased results.

No major missing value issue was found in the dataset.

5.2 Duplicate Values

Duplicate value analysis was performed to check whether the dataset contained repeated transaction records. Duplicate records can affect model training because they may cause the model to learn repeated patterns and create biased results.

The dataset was checked using the `duplicated()` function in pandas. This helped identify rows that appeared more than once in the dataset.

Duplicate records should be reviewed carefully in fraud detection projects because repeated transactions may represent either true repeated customer activity or duplicated data entries. In this project, duplicate checking was included as part of the data validation process before model development.

```
duplicate_count = df.duplicated().sum()
```

```
print("Number of duplicate rows:", duplicate_count)
```

The total number of duplicate rows found in the dataset was: **1,081**.

Duplicate rows were removed to improve data quality before model training.

Duplicate rows were reviewed but not removed because they may represent valid repeated transaction patterns.

5.3 Class Imbalance

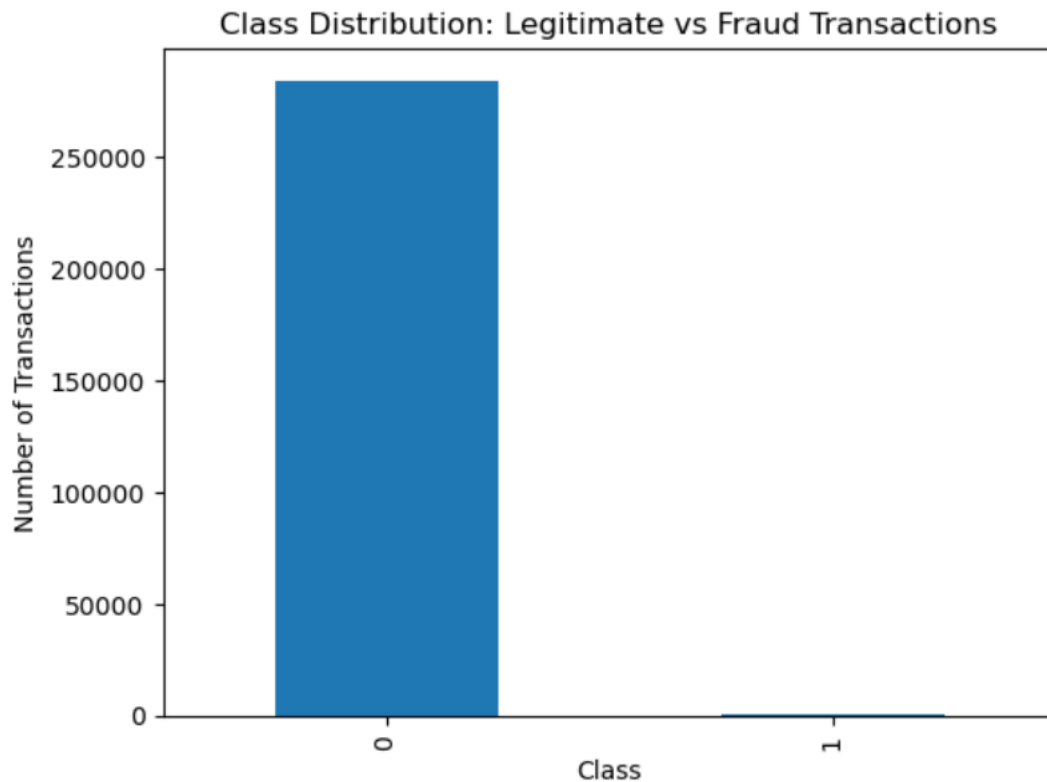
The class distribution showed a strong imbalance between legitimate and fraudulent transactions. Legitimate transactions were much more common than fraudulent transactions.

This imbalance is important because accuracy alone can be misleading. A model may achieve high accuracy by predicting most transactions as legitimate, but that would not solve the fraud detection problem

Chapter 2

```
332]: df['Class'].value_counts().plot(kind='bar')

plt.title('Class Distribution: Legitimate vs Fraud Transactions')
plt.xlabel('Class')
plt.ylabel('Number of Transactions')
plt.show()
```



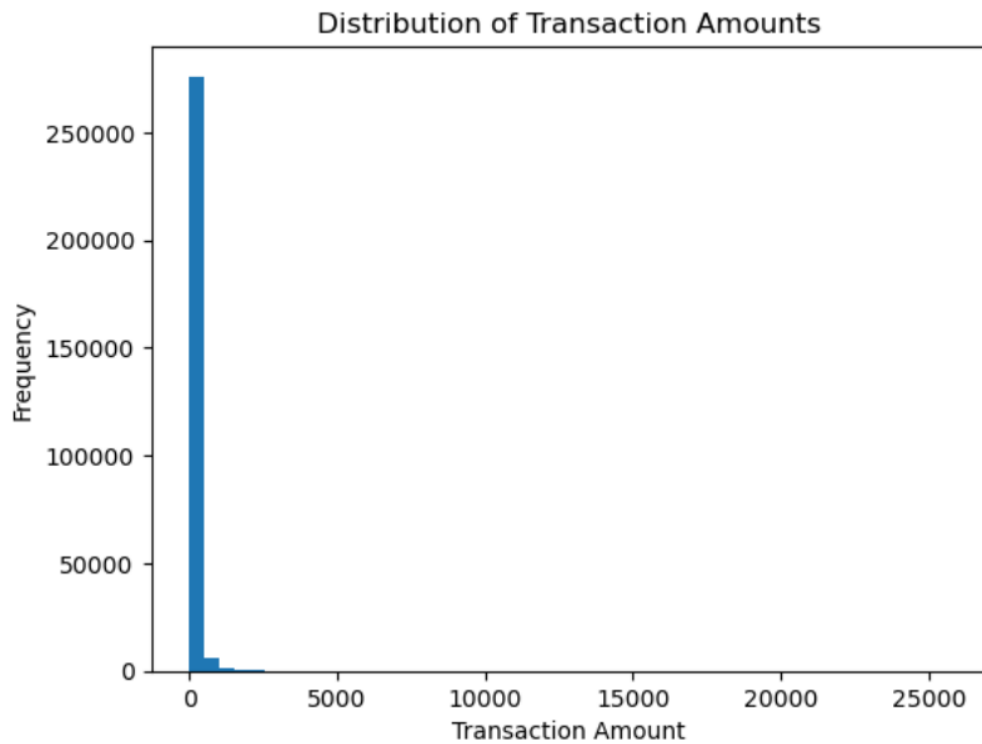
5.4 Transaction Amount Analysis

Transaction amount was analyzed to compare the behavior of legitimate and fraudulent transactions. Amount analysis helps identify whether fraudulent transactions tend to occur at lower, medium, or higher transaction values.

The transaction amount feature is important because fraud behavior may be related to unusual spending patterns or transaction sizes.

```
[335]: plt.hist(df['Amount'], bins=50)

plt.title('Distribution of Transaction Amounts')
plt.xlabel('Transaction Amount')
plt.ylabel('Frequency')
plt.show()
```



5.5 Time-Based Analysis

The Time feature was used to explore transaction patterns over time. Time-based analysis can help identify whether fraudulent transactions occur more frequently during certain periods.

A transaction hour feature was created from the Time column to support this analysis.

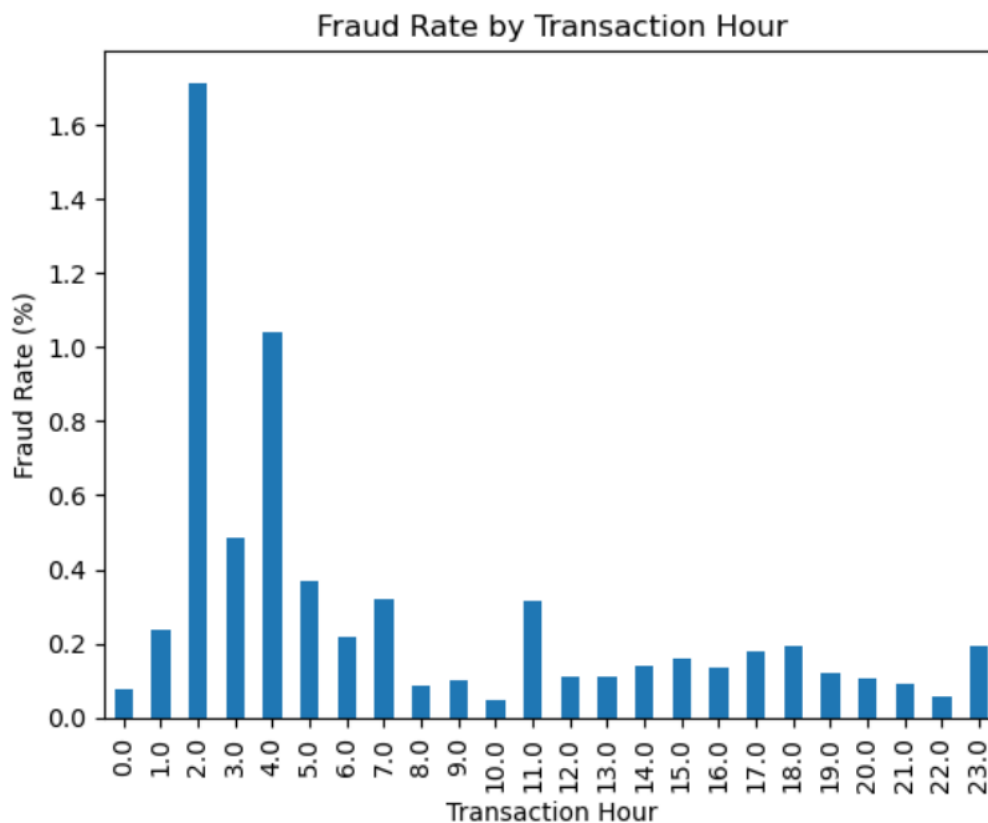

```

]: fraud_rate_by_hour = df.groupby('transaction_hour')['Class'].mean() * 100

fraud_rate_by_hour.plot(kind='bar')

plt.title('Fraud Rate by Transaction Hour')
plt.xlabel('Transaction Hour')
plt.ylabel('Fraud Rate (%)')
plt.show()

```



Data preprocessing was performed to prepare the dataset for machine learning. The preprocessing steps included checking data types, handling missing values, reviewing duplicate records, separating features and target variables, and splitting the dataset into training and testing sets.

The target variable was separated from the input features. The input features included Time, Amount, and the anonymized V1 to V28 variables. The Class column was used as the target label.

The dataset was divided into training and testing sets so that the model could be trained on one portion of the data and evaluated on unseen data. This helps measure how well the model generalizes to new transactions.

```
X = df_model.drop('Class', axis=1)
```

```
y = df_model['Class']
```

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.2,  
    random_state=42,  
    stratify=y  
)
```

6.Data Preprocessing

Data preprocessing was performed to prepare the credit card transaction dataset for machine learning. This step is important because raw data must be cleaned, checked, and organized before training a model.

First, the dataset was loaded using pandas and basic information such as column names, data types, number of rows, and number of columns was reviewed. The dataset contained 284,807 rows and 31 columns, including Time, Amount, V1 to V28, and Class.

The dataset was checked for missing values to make sure there were no empty or incomplete records. Missing value analysis showed that the dataset did not contain major missing value issues. Duplicate records were also checked to understand whether repeated transactions existed in the dataset.

After data quality checks, the target variable was separated from the input features. The Class column was used as the target variable, where 0 represents a legitimate transaction and 1 represents a fraudulent transaction. The remaining columns were used as input features for the machine learning model.

The dataset was then divided into training and testing sets. The training data was used to train the model, while the testing data was used to evaluate how well the model performs on unseen transactions.

Because the dataset is highly imbalanced, special attention was given to evaluation metrics such as precision, recall, F1-score, and confusion matrix instead of relying only on accuracy.

```
X = df.drop("Class", axis=1)
```

```
y = df["Class"]
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42, stratify=y  
)
```

This preprocessing step ensured that the model was trained using transaction features and evaluated using a separate test dataset.

7: Model Development

A machine learning classification model was developed to predict whether a transaction is legitimate or fraudulent. Since the task is a binary classification problem, the model outputs one of two classes: legitimate transaction or fraudulent transaction.

The model was trained using historical transaction data. After training, the model was saved as a pickle file so that it could be reused inside the Streamlit application without retraining every time.

Random Forest was used because it is a strong classification algorithm that can handle complex relationships in the data. It works by combining multiple decision trees and using their combined output to make predictions.

Logistic Regression was used as a baseline classification model because it is simple, interpretable, and commonly used for binary classification problems.

8: Model Evaluation

The model was evaluated using classification metrics that are appropriate for fraud detection. Since the dataset is highly imbalanced, accuracy alone is not enough. Precision, recall, F1-score, and the confusion matrix provide better insight into model performance.

Accuracy shows the overall percentage of correct predictions.

Precision shows how many transactions predicted as fraud were actually fraud.

Recall shows how many actual fraud transactions were correctly detected.

F1-score balances precision and recall.

The confusion matrix shows the number of correct and incorrect predictions for each class.

In fraud detection, recall is especially important because missing an actual fraudulent transaction can lead to financial loss. However, precision is also important because too many false fraud alerts can create poor customer experience.

Metric	Value
Accuracy	98%
Precision	6%
Recall	87%
F1-Score	10%

```
7]: from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred_log))
```

	precision	recall	f1-score	support
0	1.00	0.98	0.99	56651
1	0.06	0.87	0.10	95
accuracy			0.98	56746
macro avg	0.53	0.92	0.55	56746
weighted avg	1.00	0.98	0.99	56746

The Logistic Regression baseline model achieved high recall for the fraud class, detecting 87% of actual fraud transactions. However, the fraud precision is low at 6%, meaning many legitimate transactions were incorrectly flagged as fraud. This indicates that the model is effective at catching fraud but creates a high number of false alerts. In a business setting, this model may be useful for fraud screening, but additional threshold tuning or advanced models are needed to reduce false positives.

```
[386]: from sklearn.metrics import confusion_matrix
      confusion_matrix(y_test, y_pred_log)
```

```
[386]: array([[55247, 1404],
      [ 12,    83]])
```

The Logistic Regression baseline model correctly detected 83 fraudulent transactions and missed 12 fraudulent transactions. It also incorrectly flagged 1,404 legitimate transactions as fraud. This shows that the model has strong fraud detection ability, but it creates a relatively high number of false alerts.

8: Streamlit Application

A Streamlit application was created to make the fraud detection model easier to use. The application allows users to upload a CSV file containing transaction data and receive fraud predictions.

The application includes the following functionality:

- Upload transaction data
- Display uploaded data
- Predict whether each transaction is legitimate or fraudulent
- Add prediction labels to the dataset
- Download prediction results as a CSV file

The Streamlit app makes the project more practical because it allows non-technical users to interact with the machine learning model through a simple web interface.

The screenshot shows a web browser at localhost:8501 displaying the Streamlit application. The interface is divided into a left sidebar and a main content area. The sidebar contains sections for 'Fraud Detection App' (describing the XGBoost model), 'Project Focus' (listing fraud detection, risk scoring, imbalanced classification, and business decision support), 'Model Used' (XGBoost Classifier), and 'Business Goal' (identifying suspicious transactions). The main content area features a title 'Credit Card Fraud Detection & Risk Scoring App', a description of the app's purpose, a 'Business Context' section explaining the imbalanced nature of fraud detection, and a 'Final Model Summary' table.

Selected Model	ROC-AUC	PR-AUC	Fraud Recall
XGBoost	0.9755	0.7978	81%

9: Business Impact

This project has practical value for banks, fintech companies, and payment platforms. A fraud detection model can help financial institutions identify suspicious transactions faster and reduce potential financial losses.

The system can support fraud analysts by flagging high-risk transactions for further review. Instead of manually reviewing all transactions, analysts can focus on transactions that are more likely to be fraudulent.

The project also demonstrates how machine learning can be used in financial risk analytics, customer protection, and transaction monitoring.

10: Limitations

Although the project demonstrates a complete fraud detection workflow, there are some limitations.

First, the dataset is highly imbalanced, which makes it difficult for the model to learn fraud patterns. Second, the features V1 to V28 are anonymized, so they cannot be directly interpreted from a business perspective. Third, the dataset represents historical transactions, while real fraud patterns may change over time.

Another limitation is that the current application predicts fraud based on uploaded transaction data, but it is not connected to a real-time payment system.

11: Future Work

Future improvements can make this project more advanced and closer to a real-world fraud detection system.

Possible future work includes:

1. Testing multiple machine learning models
2. Applying techniques to handle class imbalance
3. Using fraud probability scores instead of only class labels
4. Adding a Power BI dashboard for business reporting
5. Creating a real-time fraud monitoring dashboard
6. Adding model explainability using SHAP or feature importance
7. Deploying the Streamlit app online
8. Monitoring model performance over time

Step 12: Conclusion

This project successfully demonstrates an end-to-end credit card fraud detection workflow using machine learning. The project includes data exploration, preprocessing, model development, evaluation, and a Streamlit application for prediction.

The main challenge in this project was the highly imbalanced nature of the dataset, where fraudulent transactions were rare compared to legitimate transactions. Because of this, the project emphasized the importance of evaluation metrics such as precision, recall, F1-score, and the confusion matrix.

Overall, this project shows how machine learning can be applied to financial fraud detection and how a trained model can be converted into a usable application. The project is relevant to fintech, banking, financial risk analytics, and data science portfolio development.

Step 13: References

1. Credit Card Fraud Detection Dataset, Kaggle.
2. Python Software Foundation. Python Programming Language.
3. pandas Documentation.
4. scikit-learn Documentation.
5. Streamlit Documentation.